

SPRING BOOT - MAVEN



UP ASI
Office E204

Course map

- Spring Boot Definition
- Spring Boot Benefits
- Maven Definition
- Project building steps
- Project tree
- Dependency management and scope
- TP Spring Boot

SPRING BOOT

- **Spring Boot** is the main project of the Spring Framework.
- It simplifies the start-up and development of new Spring applications.
- It dramatically reduces development time and increases productivity.
- With Spring Boot, Spring configurations are reduced.
- It is very easy to integrate Spring Boot applications with its Spring ecosystems (Spring projects) such as Spring MVC, Spring Data, Spring Security etc...

SPRING BOOT

- Spring Boot supports embedded containers.
- This allows web applications to run without deployment on a web server.
- It follows the "Default Configuration" approach to reduce development time and effort.
- Spring Boot supports work with microservices.

SPRING BOOT ADVANTAGES

- Spring Boot offers three essential benefits:
 - Configuration management
 - The server is embedded
 - Dependency management

SPRING BOOT BENEFITS - Configuration Management

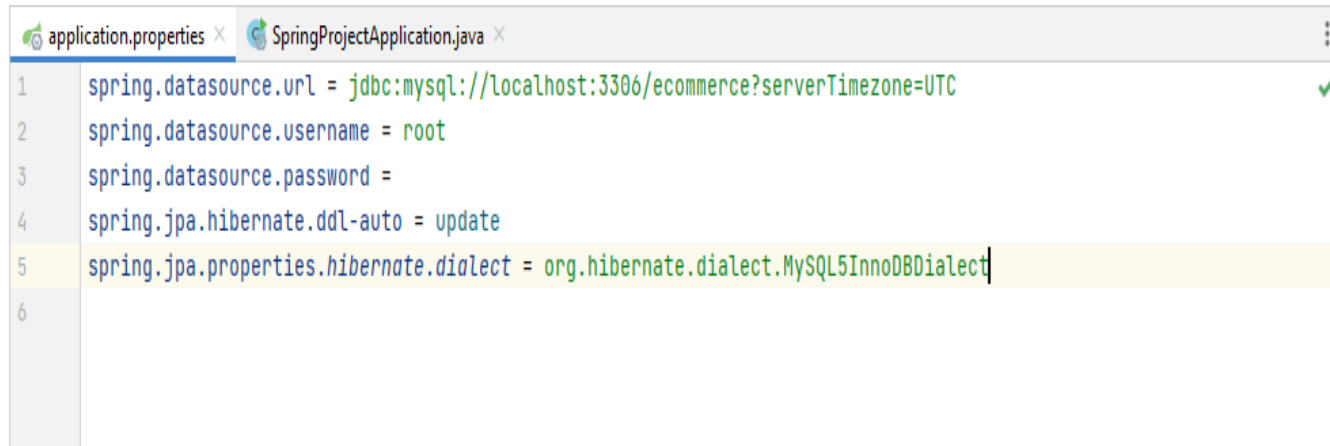
- **Spring Boot** makes **configuration management** easier by centralizing configurations in a single file. Thus by focusing on the business instead of the configuration, the developer becomes much more productive.

Before Spring Boot

web.xml
persistence.xml
dispatcher.xml
log4J.xml
.....

With Spring Boot

A single application.properties file

A screenshot of an IDE window showing the 'application.properties' file. The file contains the following configuration: spring.datasource.url = jdbc:mysql://localhost:3306/ecommerce?serverTimezone=UTC, spring.datasource.username = root, spring.datasource.password =, spring.jpa.hibernate.ddl-auto = update, and spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.MySQL5InnoDBDialect. The code is color-coded, and there is a green checkmark in the right margin next to the first line. The IDE tabs show 'application.properties' and 'SpringProjectApplication.java'.

SPRING BOOT BENEFITS - Embedded Server

- **Spring Boot** provides embedded servers (Embedded HTTP servers) like Tomcat, Jetty in order to develop and test web applications easily.
- By launching the project, the Tomcat jar unzip and launch.

The screenshot shows an IDE window with the file `springBootProject/pom.xml` open. The **Dependency Hierarchy [test]** panel is active, displaying a tree of dependencies. The **Resolved Dependencies** panel is also visible, showing a list of resolved dependencies. The **Filter** is set to `tomcat`.

Dependency Hierarchy [test]

- spring-boot-starter-web : 2.3.4.RELEASE [compile]
 - spring-boot-starter-tomcat : 2.3.4.RELEASE [compile]
 - tomcat-embed-core : 9.0.38 [compile]
 - tomcat-embed-websocket : 9.0.38 [compile]
 - tomcat-embed-core : 9.0.38 (omitted for conflict with 9.0.38)

Resolved Dependencies

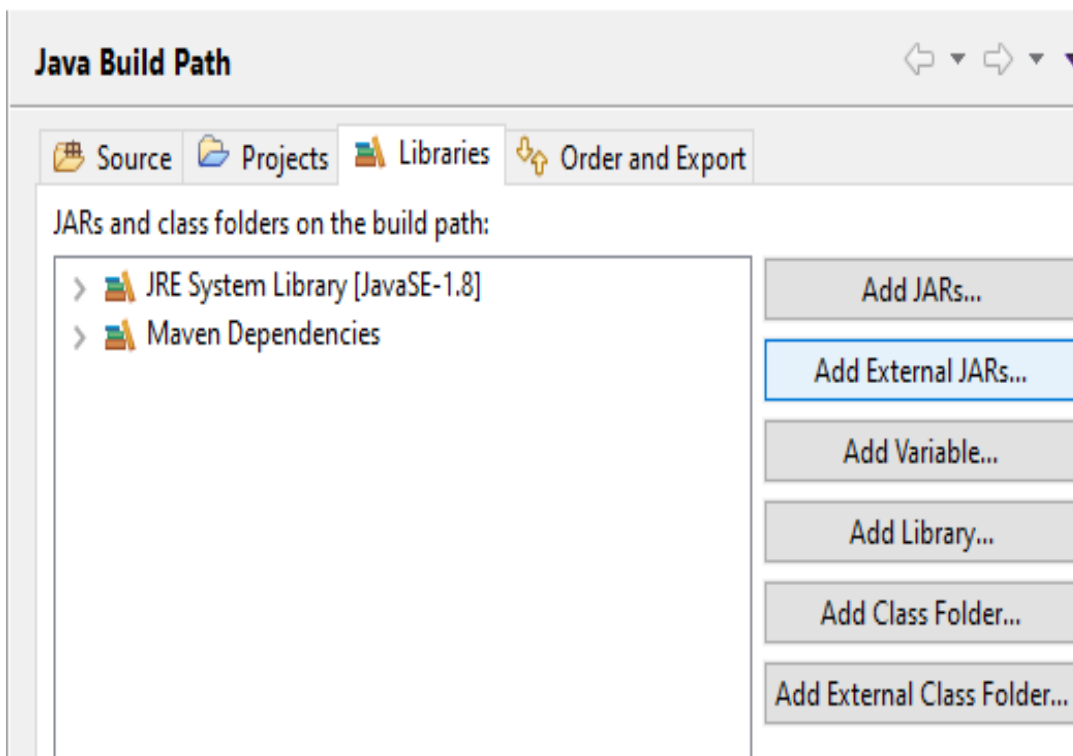
- mysql-connector-java : 8.0.21 [runtime]
- objenesis : 2.6 [test]
- opentest4j : 1.2.0 [test]
- slf4j-api : 1.7.30 [compile]
- snakeyaml : 1.26 [compile]
- spring-aop : 5.2.9.RELEASE [compile]
- spring-aspects : 5.2.9.RELEASE [compile]
- spring-beans : 5.2.9.RELEASE [compile]
- spring-boot : 2.3.4.RELEASE [compile]
- spring-boot-autoconfigure : 2.3.4.RELEASE [compile]
- spring-boot-starter : 2.3.4.RELEASE [compile]
- spring-boot-starter-aop : 2.3.4.RELEASE [compile]
- spring-boot-starter-data-jpa : 2.3.4.RELEASE [compile]
- spring-boot-starter-jdbc : 2.3.4.RELEASE [compile]
- spring-boot-starter-json : 2.3.4.RELEASE [compile]

Overview | Dependencies | **Dependency Hierarchy** | Effective POM | pom.xml

SPRING BOOT BENEFITS - Dependency Management

- Spring Boot makes it easy to manage dependencies to start a Spring project.

Before Spring and Spring Boot



With Spring Boot

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

SPRING BOOT BENEFITS - Dependency Management

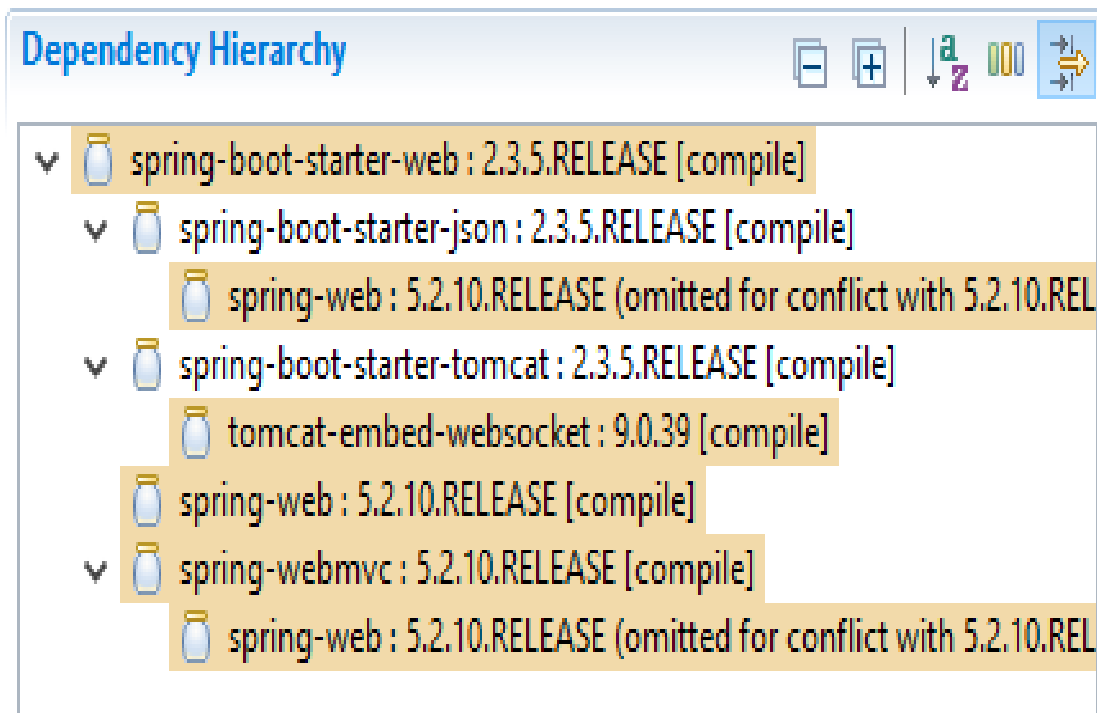
- Spring Boot facilitates **dependency management** through the use of **starters**.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
```

SPRING BOOT BENEFITS - Dependency Management

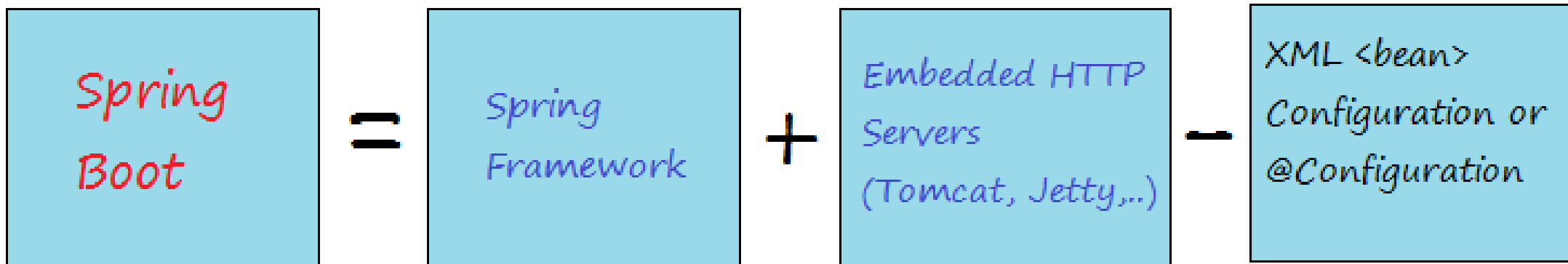
- A **starter** will bring to your project a set of dependencies, commonly used for a given type of project.
- Starters make **version management** easier. No need to search for compatible versions to add them one by one in pom.xml.



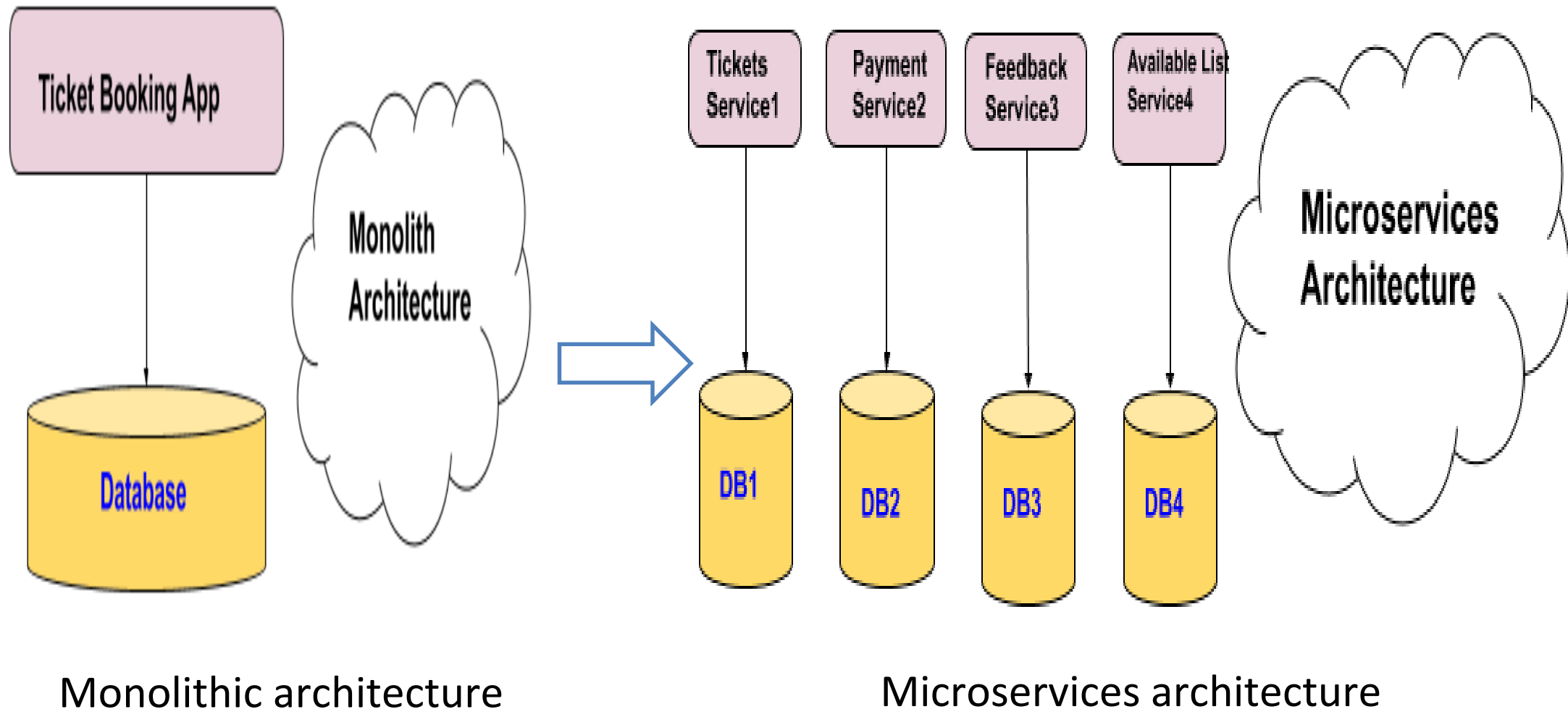
```
<parent>
<groupId>
org.springframework.boot
</groupId>
<artifactId>
spring-boot-starter-parent</artifactId>
<version>2.7.3.RELEASE</version>
<relativePath/>
</parent>
```

SPRING BOOT BENEFITS

Spring Boot can be explained simply by the illustration below:

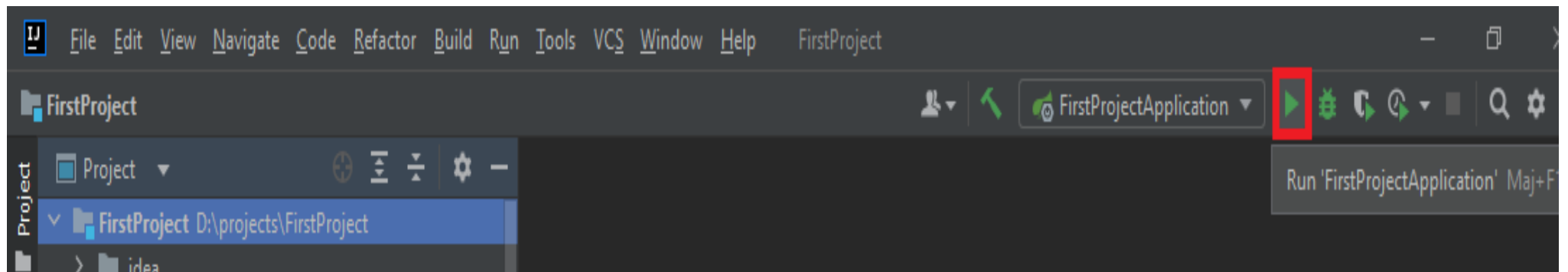


Spring Boot and microservices (optional)



SPRING BOOT

- **How to start a Spring Boot project:**



- Spring Boot provides many plugins to develop and test Spring Boot applications quickly using Build tools like Maven and Gradle.

MAVEN DEFINITION

Maven is a tool for the **management** and **automation** of project production (**project construction**) developed by the Apache Foundation allowing:

- Continuous integration
- Managing local and remote dependencies in the Convention-Based Project Model (POM)
- Automate build management and deliverable generation
- Automation of recurring tasks
- The launch of tests



MAVEN INSTALLATION

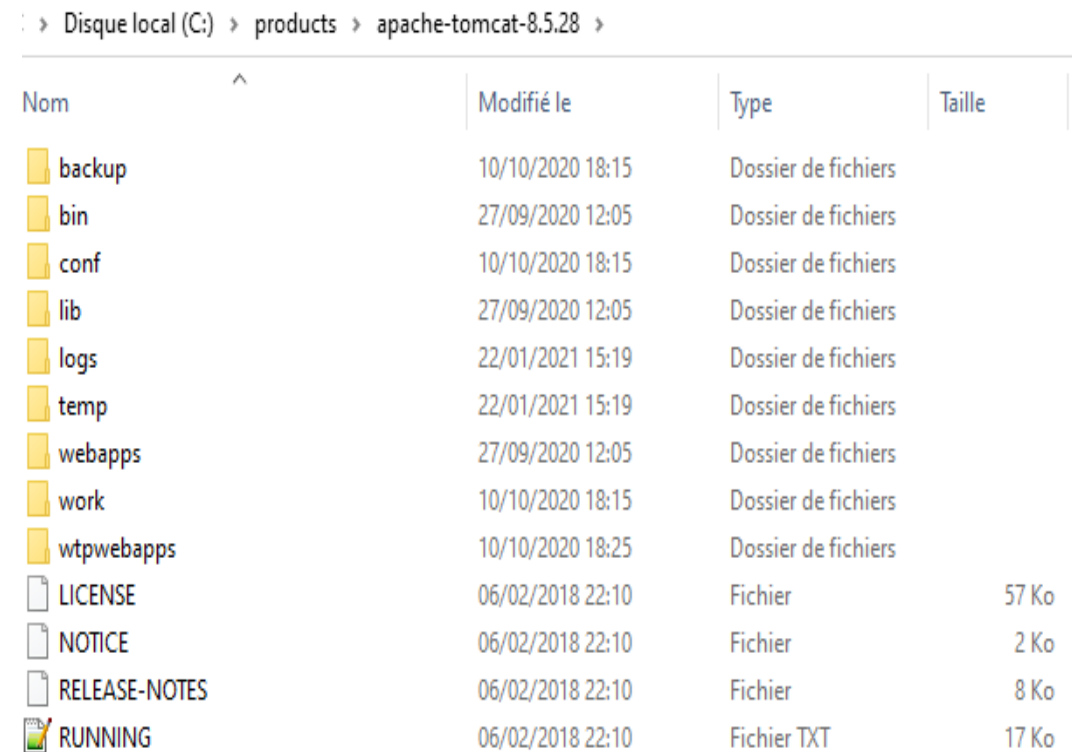
- Maven can be installed:

- In standalone mode

Example: Using Maven in continuous integration (DevOps domain)

- **As a plugin**

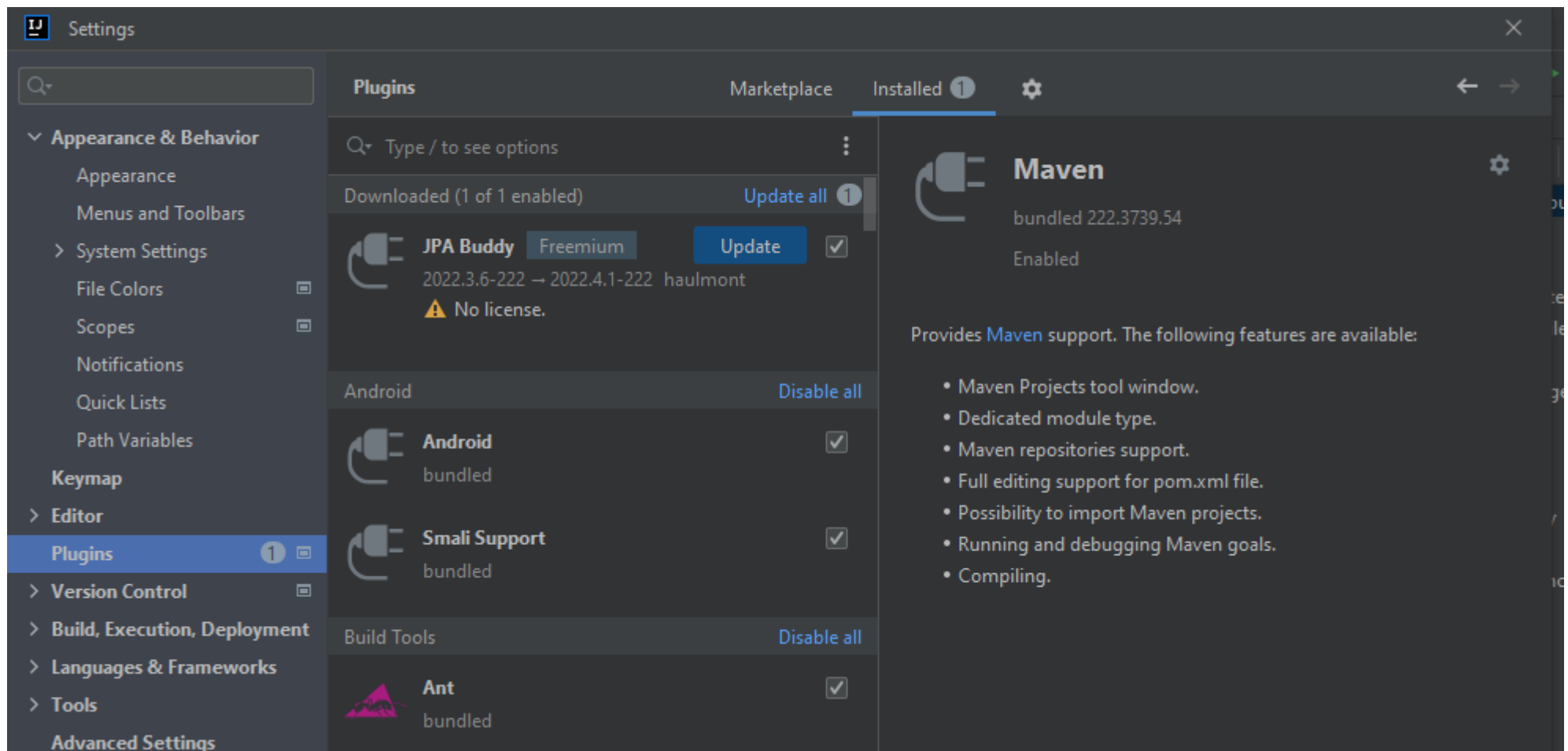
By default, IntelliJ integrates a Maven plugin



Disque local (C:) > products > apache-tomcat-8.5.28				
Nom	Modifié le	Type	Taille	
backup	10/10/2020 18:15	Dossier de fichiers		
bin	27/09/2020 12:05	Dossier de fichiers		
conf	10/10/2020 18:15	Dossier de fichiers		
lib	27/09/2020 12:05	Dossier de fichiers		
logs	22/01/2021 15:19	Dossier de fichiers		
temp	22/01/2021 15:19	Dossier de fichiers		
webapps	27/09/2020 12:05	Dossier de fichiers		
work	10/10/2020 18:15	Dossier de fichiers		
wtpwebapps	10/10/2020 18:25	Dossier de fichiers		
LICENSE	06/02/2018 22:10	Fichier	57 Ko	
NOTICE	06/02/2018 22:10	Fichier	2 Ko	
RELEASE-NOTES	06/02/2018 22:10	Fichier	8 Ko	
RUNNING	06/02/2018 22:10	Fichier TXT	17 Ko	

MAVEN INSTALLATION

- In plugin mode:
- Click on File => Settings => plugins



MAVEN FIRST PROJECT

A project is characterized by:

- **project** : Root tag of all pom.xml.
- **modelVersion** : Version of POM used.
- **groupId** : Identify a group that created the project. Ex: org.apache.
- **artifactId** : Unique name used to name the artifact to be built.
- **packaging** : Type of project packaging (ex. : JAR, WAR, EAR...).
- **archetype** : Project template.
- **name** : Project Name.
- **description** : Project description.

Project Release Management

Exercise

Either the following project versions:

1.0.0

1.1.0

2.0.0

2.0.1

3.0.0

What is each numbering?

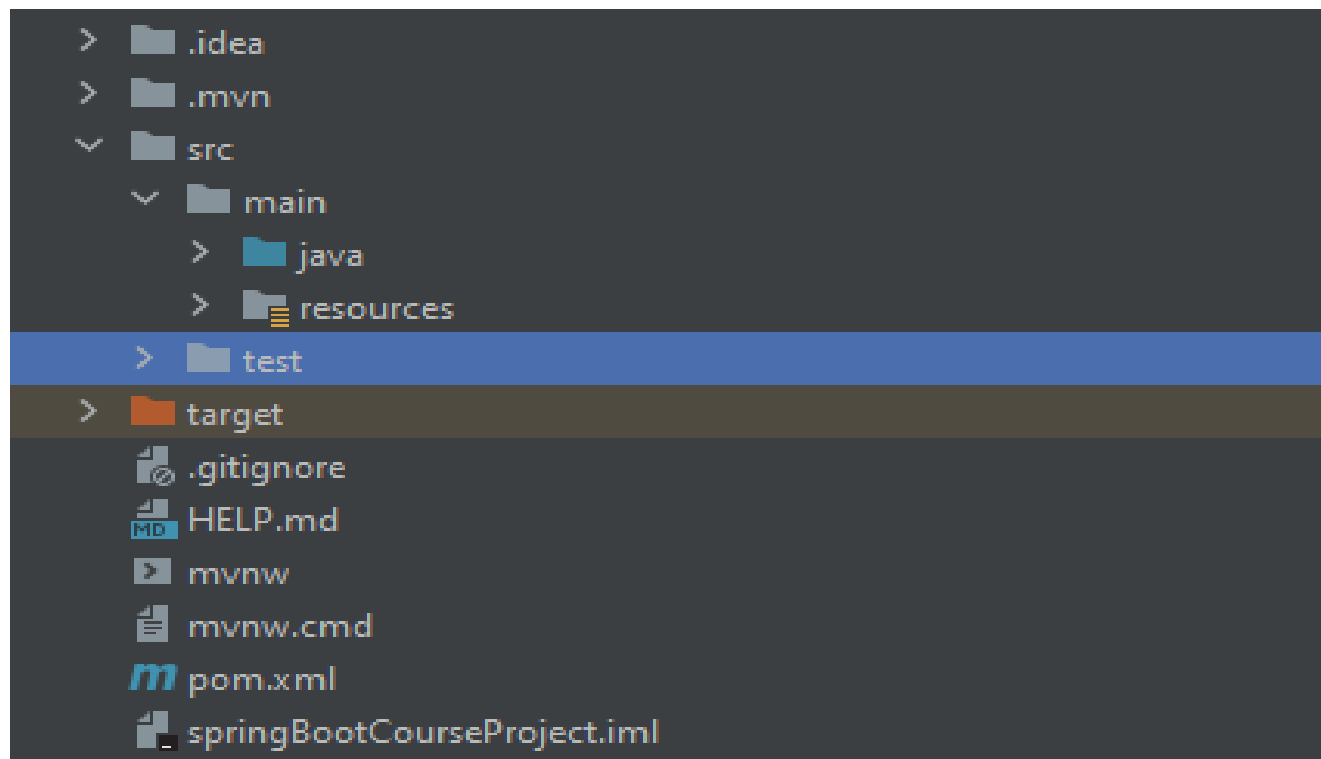
Major release

Minor release

Patch

The project tree

- Maven is based on the concept of **convention over configuration** (predefined tree) which differentiates it from its competitors (Gradle, Ant) where additional configuration is required.



The project tree

- **pom.xml** : the project configuration file
- **/src** : source code and main source files
- **/src/main/java** : Java source code
- **/src/main/resources** : resource files (images, config files...)
- **/src/main/webapp** : project webapp
- **/src/test** : test files
- **/src/test/java** : test java source code
- **/src/test/resources** : test resource files
- **/target** : result files, binaries (of code and tests), generated packages and test results

POM.XML file

- Specifies the dependencies the project needs.
- Spécifie l'emplacement de l'artefact du projet (groupId, ArtifactId, version)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.esprit.spring</groupId>
<artifactId>springBootCourseProject</artifactId>
<version>1.0.0</version>
<name>springBootCourseProject</name>
<description>springBootCourseProject</description>
<dependencies><dependency>
<groupId>org.apache.logging.log4j</groupId>
<artifactId>log4j-core</artifactId>
<version>2.14.0</version>
</dependency></dependencies>
</project>
```

Project construction stages

- **mvn compile** : Create the . class
- **mvn test** : Play the unit tests
- **mvn package** : Creation of the deliverable in target.
- **mvn install** : Copy of the deliverable in the local Repository:
~\.m2\repository\...
- **mvn deploy** : Copy of the deliverable to the remote repository
- **mvn clean** : Deletes the contents of the target folder.

Project construction stages

- Deliverable location:
 {repository location}/groupId/artifactId/version
- Package name (jar in general) : {artifactId}-{version}.{package}

Ce PC > Disque local (C:) > Utilisateurs > ASUS > .m2 > repository > com > esprit > spring > springBootCourseProject > 1.0.0

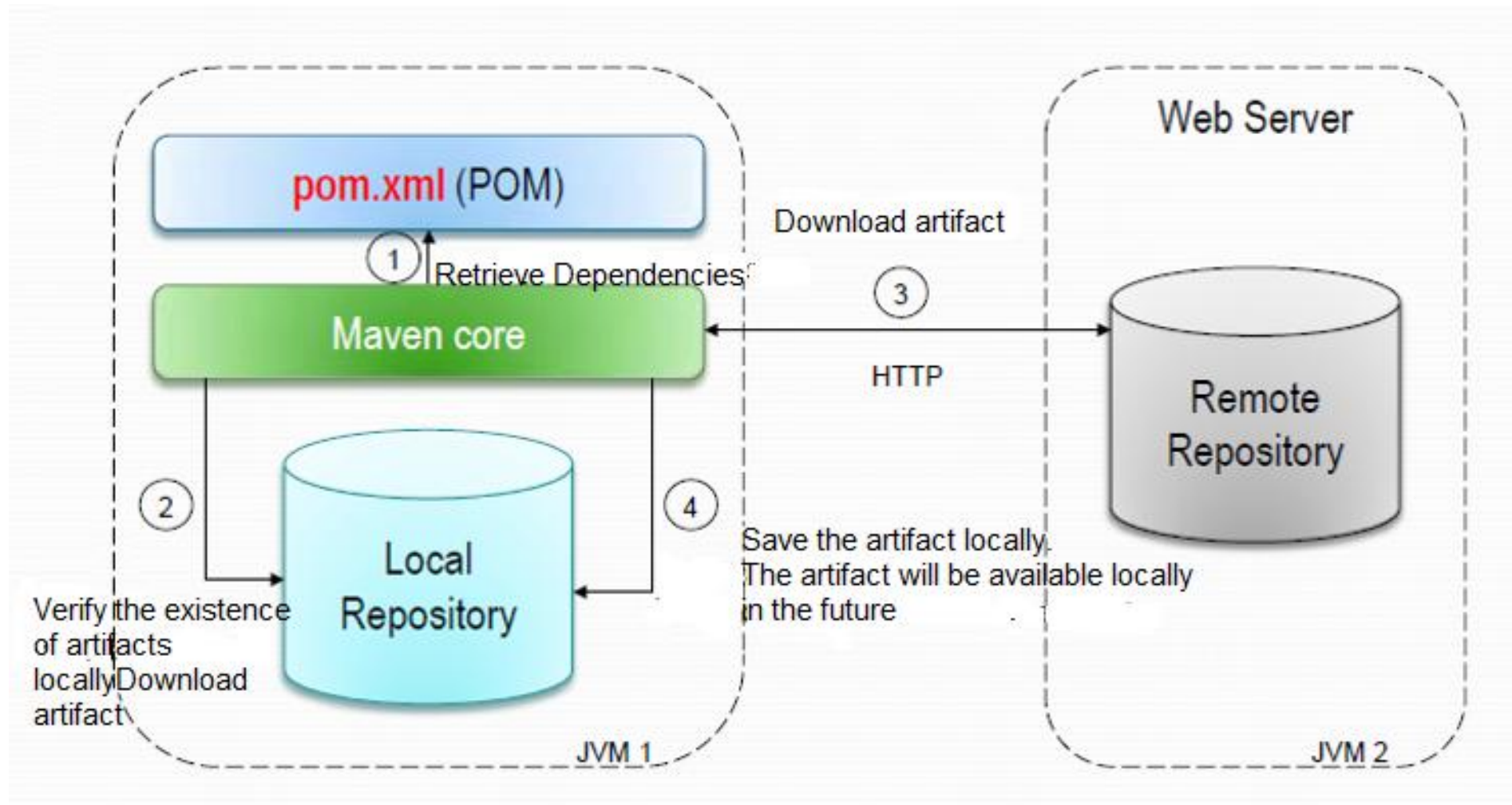
	Nom	Modifié le	Type	Taille
	 _remote.repositories	14/09/2022 07:29	Fichier REPOSITOR...	1 Ko
	 springBootCourseProject-1.0.0	14/09/2022 07:29	Executable Jar File	38 016 Ko
	 springBootCourseProject-1.0.0.pom	14/09/2022 07:29	Fichier POM	3 Ko
				

dependency management

To add a dependency, simply look for the dependency in question in the **mvnRepository** (<https://mvnrepository.com/>) and include it in pom.xml under the tag **<dependencies>** as follows:

```
<dependencies>
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.23</version>
</dependency>
</dependencies>
```

Dependency management



Maven Deposits

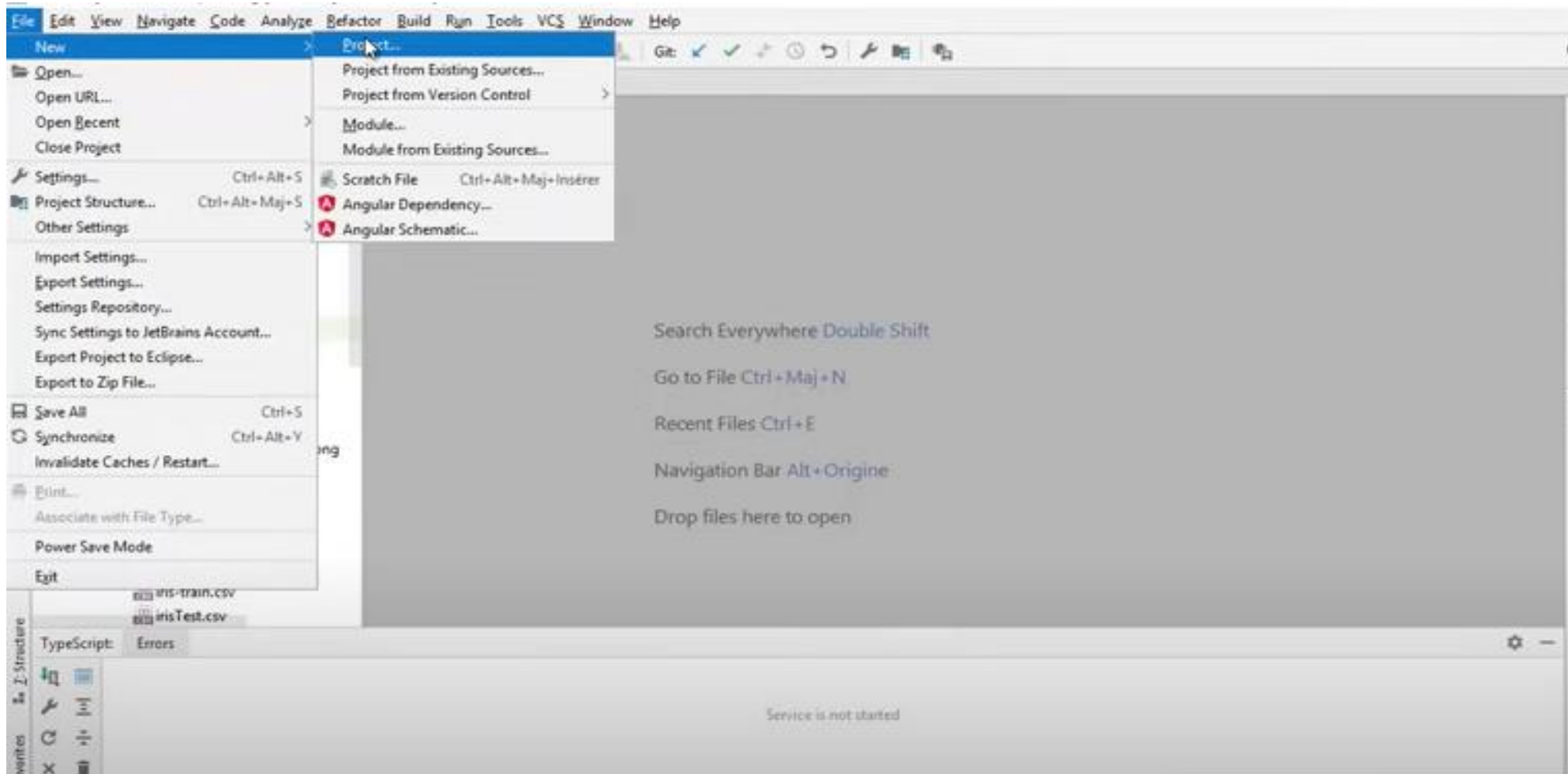
- Maven relies on repositories to store jars of dependencies and deliverables.
- There are two kinds of repositories :
 - Local: The local repository is located by default under the %path_dossier_M2%\repository
 - Remote:
 - Central : dépôt public Maven accessible through <https://mvnrepository.com/> ou
 - Internal (Private) : in the servers dedicated to the company (for security reasons)
- Repositories are organized into groups, artifacts and versions.

TP - Spring Boot- Maven

- In this tutorial we will create our first Spring Boot project.
- We will discover the different commands related to the Maven life cycle and apply them to the spring boot project already created
- It will be used in the following courses (Spring Data JPA)
- The steps will be described in the following slides:

TP - Spring Boot- Maven

- Creating of an Spring Boot project



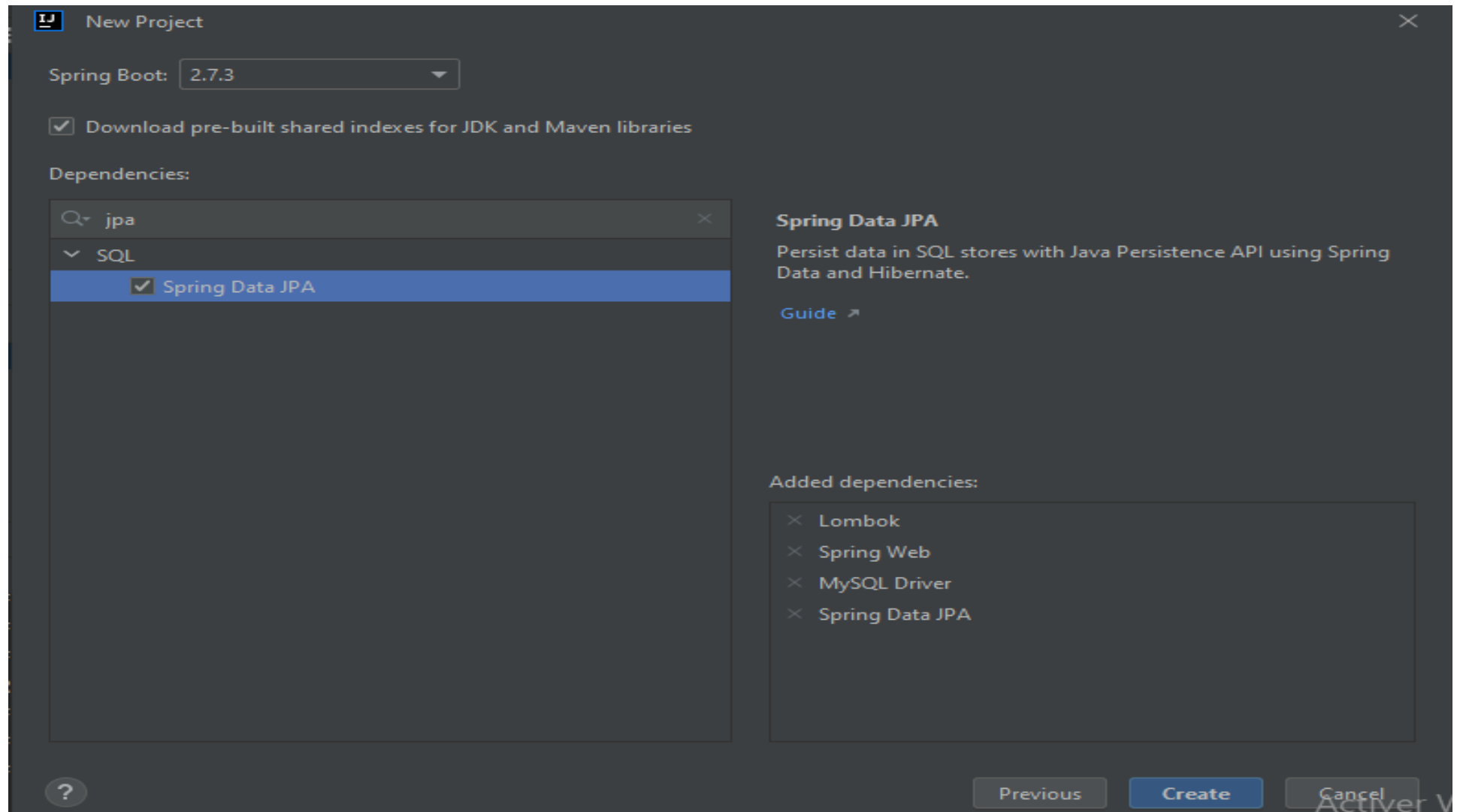
TP - Spring Boot- Maven

The screenshot shows the 'New Project' dialog in IntelliJ IDEA, configured for a Spring Boot project using Maven. The 'Spring Initializr' generator is selected in the left sidebar. The main configuration area on the right includes the following fields and options:

- Server URL:** start.spring.io
- Name:** spring-boot
- Location:** C:\Work\workspace-intellij (The project will be created in: C:\Work\workspace-intellij\spring-boot)
- ☐ Create Git repository
- Language:** Java (selected), Kotlin, Groovy
- Type:** Maven (selected), Gradle
- Group:** tn.spring
- Artifact:** spring-boot
- Package name:** tn.spring.springboot
- JDK:** 1.8 Oracle OpenJDK version 1.8.0_121
- Java:** 8
- Packaging:** Jar (selected), War

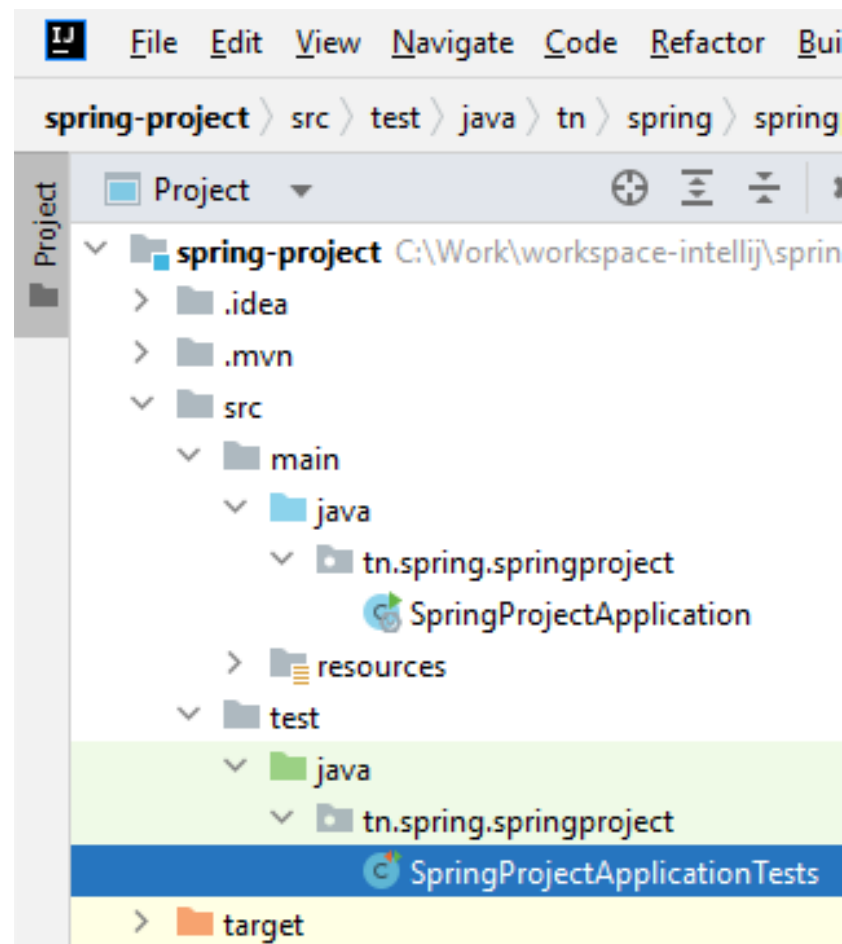
At the bottom right, there are 'Next' and 'Cancel' buttons. A help icon (?) is located at the bottom left of the dialog.

TP - Spring Boot- Maven



TP - Spring Boot- Maven

- Remove the test class to avoid errors when calling Maven commands (because "Maven install" for example will try to launch the unit tests):



TP - Spring Boot- Maven

- Add the following properties to avoid errors when launching Maven commands (As there is the Spring Data JPA dependency, Maven will check if there is a database configured):

DATABASE

```
spring.datasource.url=jdbc:mysql://localhost:3306/springdb?useUnicode=true&useJDBCCompliantTimezoneShift=true&createDatabaseIfNotExist=true&useLegacyDatetimeCode=false&serverTimezone=UTC
```

```
spring.datasource.username=root
```

```
spring.datasource.password=
```

JPA / HIBERNATE

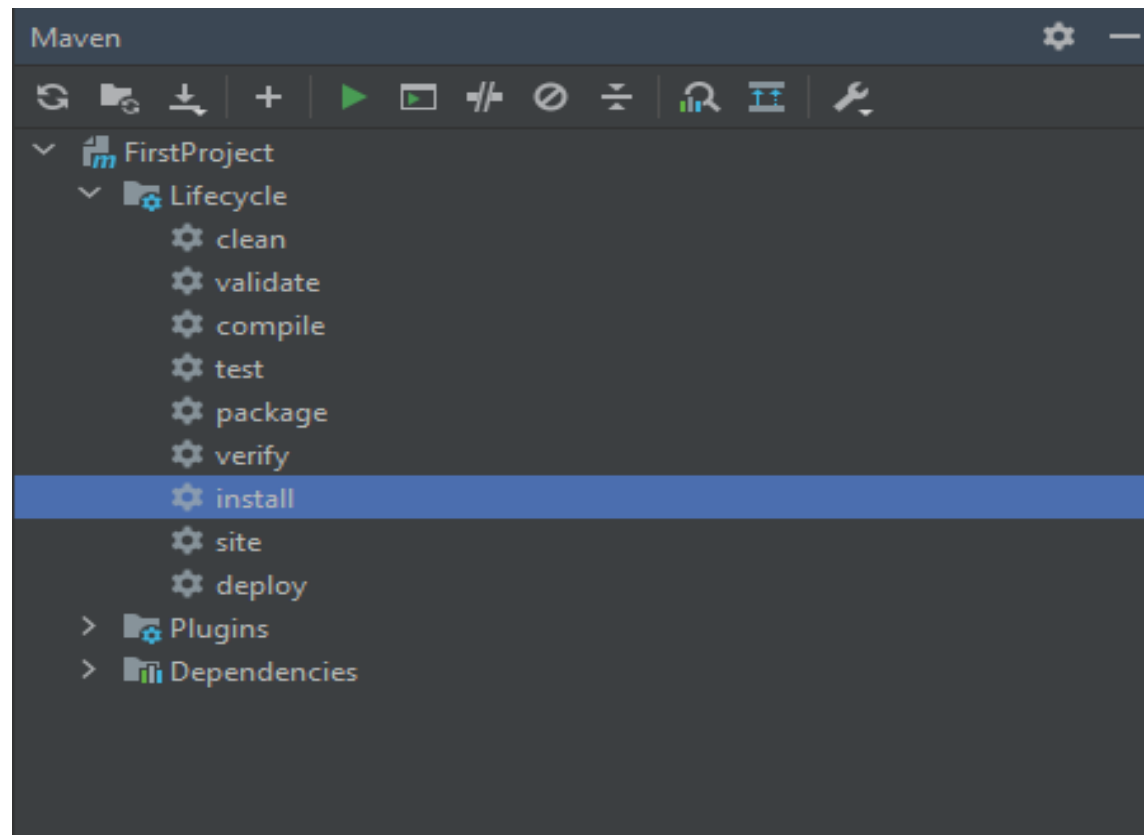
```
spring.jpa.show-sql=true
```

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
```

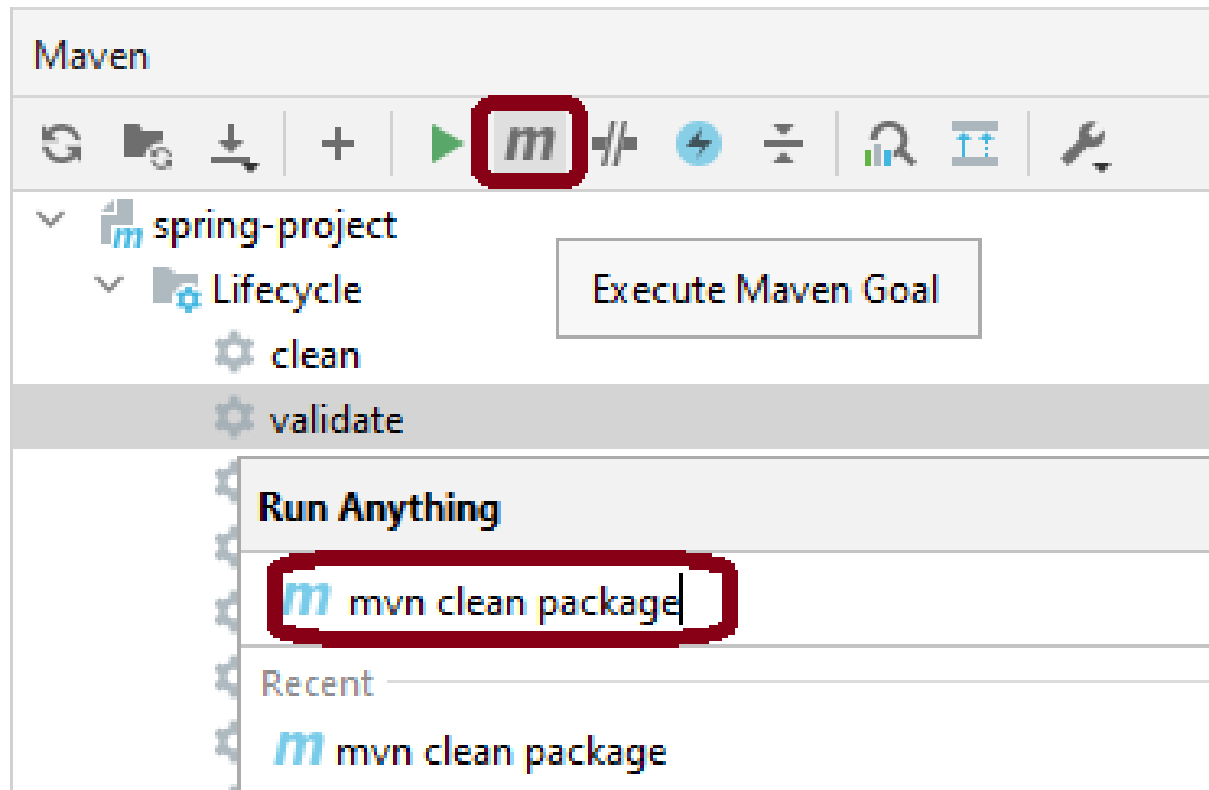
TP - Spring Boot- Maven

Test the different maven commands



TP - Spring Boot- Maven

- We can make a custom Maven command using Maven Goal: Click on the symbol "m" and type the desired command, eg mvn clean package or mvn clean install or ... :



Error (if older MySQL)

- For your information, we have added the following parameters « `?useUnicode=true&useJDBCCompliantTimezoneShift=true&useLegacyDatetimeCode=false&serverTimezone=UTC` » in the `application.properties` file to avoid the following error at runtime :
`java.sql.SQLException: The server timezone value 'Paris, Madrid' is unrecognized or represents more than one time zone. You must configure either the server or JDBC driver (via the serverTimezone configuration property) to use a more specific timezone value if you want to utilize time zone support.`
- We will see all this in detail later. Spring Boot will be the tool that will allow us to create all our next projects.

SPRING BOOT-Maven

If you have any questions, do not hesitate to contact
US:

IT department
UP ASI
Office E204